Weeks 6 to 9: build, iteration and validation

Bridge · FDU AI Experiential Learning Lab team · Track 1, Smart Healthcare

The Solution Design Guide covers weeks 1 to 5. Weeks 6 to 8 are the build and iteration phase, and week 9 is documentation and narrative. This is the record of what actually happened in those weeks, written from the commits, the Bob task sessions and the evaluation runs rather than from memory.

Weeks 6 to 8, build and iteration

How IBM Bob was used

Six tasks on the provisioned `ibm-skillsbuild-ell` instance, one per component, consuming about 5.5 of the team's 40 shared Bobcoins. Task ids, prompts and outcomes are in `bob_sessions/task_log.md`, with a session consumption screenshot for each.

Bob was not used to generate the system. It was used to attack it. The standing instruction in each task was to invent harder inputs than the ones we had thought of, then fix whatever broke, without weakening any existing safety rule.

Task	Component	Outcome
01	Interpretation and escalation	Four defects found and fixed. Ten harder reply phrasings pinned by tests.
02	Orchestrator	Dry run mode that prints the cycle plan for a given day without writing to the database.
03	Synthetic data and knowledge corpus	Type 2 diabetes added as a third condition, with its own cited guidance file.
04	Outreach content	Templates rewritten to read like a person wrote them. Caregiver variant added. Readability check added to the test suite.
05	Ranking and the nurse worklist	Open case and gone quiet filters. Evidence trail made readable on a phone.
06	Cross component regression fix	The unplanned one. Described below.

The four interpretation defects Bob found

- Urgent terms matched only exact spellings, so *cant breath* was read as non urgent.
- A past tense phrasing of a warning sign was mis-scored because only the present tense was listed, which would have under-ranked a deteriorating heart failure patient.
- Hedged replies such as *she seems alright i think* were read as reassurance rather than ambiguity.
- The ambiguity word limit was one word too tight.

None of these were failures of the model. They were failures of our own rules, found by being asked to produce inputs we had not imagined.

The regression that proves the evaluation harness is real

> After task 03 added type 2 diabetes, the harness showed **median cycles to detection rising from 1 to 4** while the headline detection rate stayed at 89 percent and concealed it.
>
> The cause was a gap between two components. The rising concern term list still covered only heart failure and COPD, so diabetes decline replies such as *blood sugar readings have been very high the last two days* scored mild instead of rising. Four of the nine seeded deterioration patients were being under-ranked.
>
> Task 06 extended the term list from that condition's own guidance file and added tests pinning every diabetes decline phrase. Median detection returned to 2.5 cycles, and genuine deterioration in the top five rose from 57 slots to 83.

This matters more than the fix. A headline number that stayed flat while the system got materially worse is exactly the failure mode an evaluation harness exists to catch, and it only caught it because latency and detection rate are measured separately.

Testing and validation

The suite grew from 23 tests to 56 across the six tasks, all passing. It runs in under five seconds from a clean clone with no third party packages.

Several tests exist to pin safety properties rather than behaviour. One fails if `close_case` ever appears in the escalation agent's source, which is how the rule that the system never closes a case is enforced structurally rather than by convention.

IBM watsonx.ai, switched on and measured

The interpretation agent sends every non urgent reply to Granite for a second opinion. The constraint is enforced in code: Granite may raise a concern level, never lower one, and is never asked to resolve an ambiguous reply.

`bridge/llm.py` deliberately swallows errors so a failed call degrades to the rule engine rather than stopping a cycle. That is correct at run time and useless afterwards, because a run could report using watsonx while having silently fallen back on every single reply. We added call accounting and a connectivity check that fails loudly, so the claim is checkable.

Measure	Rule engine	With Granite
Detection rate	89 percent, 8 of 9	100 percent, 9 of 9
Median cycles to detection	2.5	2
Worst case cycles to detection	5	5
Top five slots on real deterioration	83 of 145	90 of 145
Unexplained top five slots	5	5
Deferred urgent replies	0	0
watsonx.ai calls	none	547 attempted, 547 answered, 0 fell back

The seeded event the rule engine missed was a COPD patient who wrote *More out of breath getting dressed than I was last week*. No term in the rising concern list matched it, so the rules scored it mild. Granite read the comparison with last week and scored it rising, three days before the rules reached the same conclusion from a later and more explicit reply.

Note what did not move. Unexplained top five slots stayed at 5 and deferred urgent replies stayed at 0. Granite caught more without crowding the list, which is what the raise-only constraint exists to protect.

Model availability changed during the build: `ibm/granite-3-8b-instruct` was retired from us-south. `scripts/check_watsonx.py` verifies the connection and lists what the region actually serves, so this does not silently become a rule engine run for the next person who clones it.

What we did not do

IBM watsonx Orchestrate was not used. The seven agents and the daily cycle are defined and run in `bridge/orchestrator.py`, which keeps the prototype free of a cloud dependency. Orchestrate is where this cycle would be hosted in a real deployment. We would rather state that plainly than claim a deployment that does not exist.

Week 9, documentation and narrative

- Public repository, cloning and running with one command, verified from a fresh anonymous clone.
- README carrying both evaluation runs and the comparison between them.
- Problem and solution statement, 490 words. Technology statement, 487 words. Both within the 500 word limit.
- Three minute demo script with shot list, narration and the numbers to quote.
- Six Bob task session records in `bob_sessions/`.
- Both evaluation runs committed as JSON, so an evaluator can check the comparison without rerunning anything.

Two corrections made during documentation

Writing the documentation surfaced two numbers that had gone stale after the regression fix and were still being quoted:

- The problem and solution statement claimed a median of one cycle from first signal to case raised. The true figure is 2.5. Corrected.
- The README results table still carried pre-regression figures, including 0 of 1111 ranked entries and a worst case of 4 cycles. Corrected against a fresh run.

Both would have been false claims in a graded submission. They are recorded here because catching them is part of the work, not an embarrassment to hide.

Every figure in this document comes from a committed evaluation run, `out/eval_results_rules.json` or `out/eval_results_granite.json`, in `github.com/likithaseepelly/bridge-ibm-skillsbuild`. Nothing here is estimated.